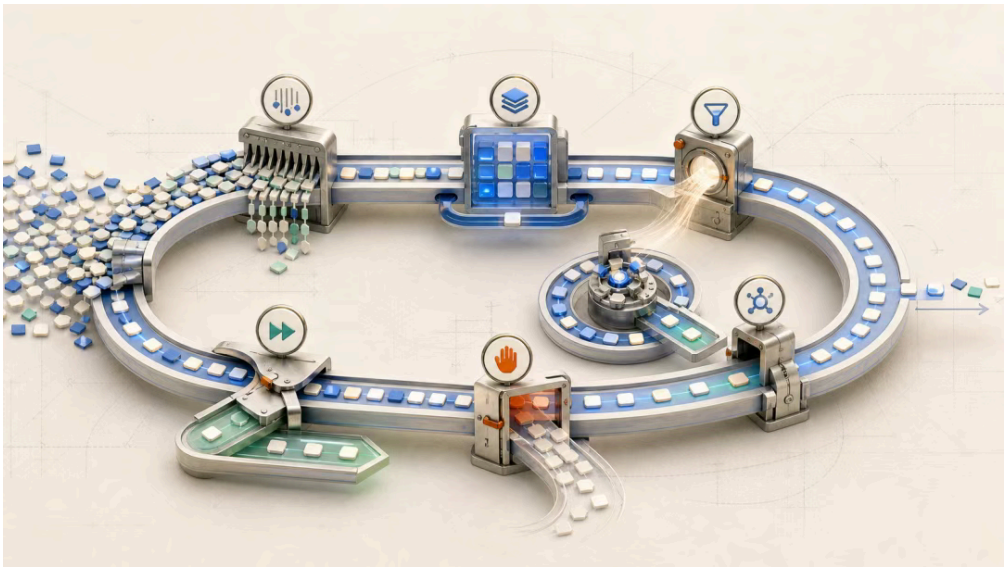
Agent 终章（Harness 成本篇）：一次百炼账单降低 88% 实战

原创 景钊 阿里技术 2026年8月21日 03:18 浙江 44人



这是2026年的第 54 篇文章

（ 本文阅读时间：约 20 分钟 ）

AI 编码半小时就可以完成一个 agent，但是一个没做过任何优化的 agent，跑一轮就是几十 M token，月底一看，费用爆炸。眼下，token 的使用已经进入精细化时代。目前个人用 token 不少公司已经开始设限，下一步大概率是开始梳理各类 agent ，在 agent 逐渐同质化下最后就是胜者为王，谁用最少的钱跑出最好效果。因此，优化 token 不是锦上添花，是 agent 能不能活下去的前提。

本文借鉴 oh-my-pi(token优化激进派)、Codex CLI、Grok Build 与 Anthropic Agent 工程笔记，结合百炼 qwen3.7-max 生产实践而成。

最后的效果：隐式缓存命中率从 33% 拉到 80%，显式缓存加权命中近九成，一轮 20 次的会话，输入费用降到原来的十分之一。

任何优化都得有依据，本文依旧如此。因此开始之前需要对齐两件事：token 花在了哪里，以及怎么量。

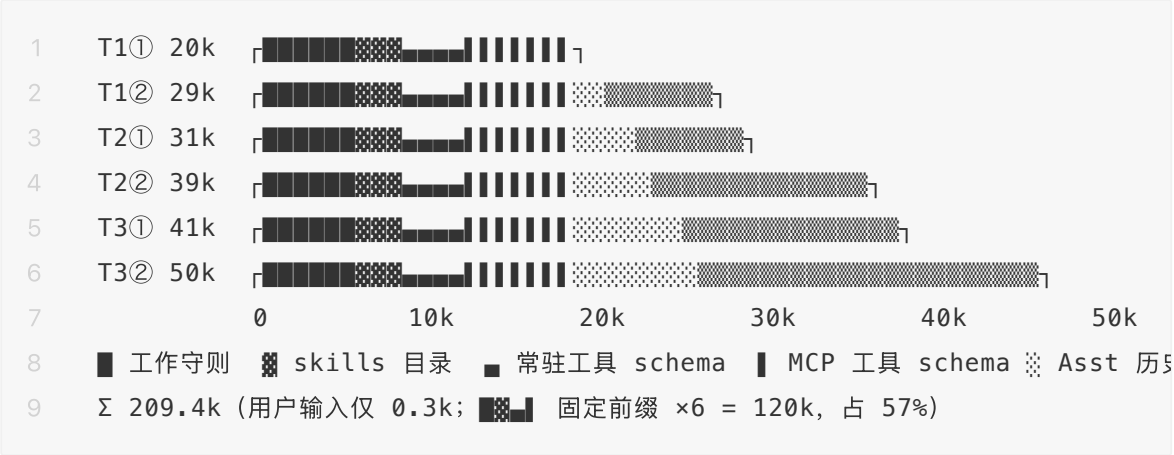
01
第一步：为什么几个字能撬动几十 M Token

一次 prompt 通常发给 agent 的往往只有几个字——“帮我看下这个应用为什么报错”。实际上，这几个字只是“冰山露出水面”的那一角，水面下才是大头：工作守则（system prompt）、几十个工具的 schema、每一轮的对话历史、工具查回来的结果。以我们做的故障诊断、成本分析 agent 为例，一个模糊的需求，agent 会铺开多个维度找线索，再拿着线索查根因，一来一回就是 30-40 轮，烧掉几十 M token。同时，由于模型没有记忆，要让它记得上一轮查了什么，唯一的笨办法就是把之前的内容原样再发一遍；第一次搜索返回 8K token，下一轮就得把这 8K 当输入带上。于是水面下越滚越大：循环每多转一圈，历史和工具结果就再叠一层。

缓存命中率优化

几个字撬动几十 M token，其实就三件事：循环反复跑、历史每轮全量重发、工具结果不断往上下文里堆。

下面用三轮对话示意图：



02
第二步：建立度量标准

优化需要验收标准，否则就没有数据支撑。我们通过三条链路交叉验证：

1、 自研。通过采集每次 LLM API 返回的多个 Token 字段，同时写入本地。`cache_hit_rate = cached_tokens / prompt_tokens`，接口未返回 usage 时记为空而非 0。`cache_mode` 从响应推断：有 `cache_type: "ephemeral"` 为显示缓存，有 `cached_tokens` 无 `cache_type` 为隐式缓存

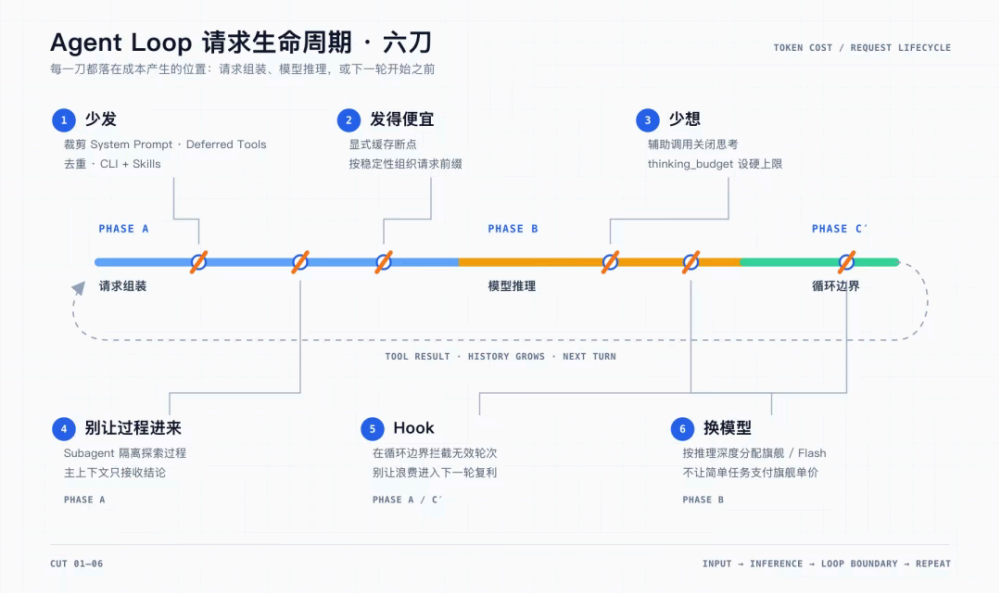
2、自建Langfuse。使用 Langfuse sdk 进行统计：同一次调用上报为一条 Generation（一个 Agent 任务 = 一条 Trace，Subagent 独立 Trace 通过 `parent_task_id` 跳回父级）。`usage_details` 按 OpenAI completion schema 拆分，服务端自动扣缓存计费；并且走的是异步队列，不阻塞主 loop。

3、自建评测 Agent。任务结束后，先从工具轨迹中标记重复调用、连续失败等异常，再由轻量模型从相关性、工具效率、重复操作、错误恢复和完整性五个维度评分。低分或异常任务会自动沉淀诊断包，作为后续优化 Prompt、工具和 Agent Loop 的样本，形成持续改进闭环。

质量是成本优化的基础，不能因为控制成本就降低质量。6 月与 7 月的总分基本持平；7 月又叠加了钉钉诊断任务大规模接入和优化的一些边界问题在上线后才发现，因此总分被前半月低分拉低；最后完整落地是 8 月份。



有了尺子，下一步就是找到下刀的位置。一次 agent loop 的请求生命周期里，我们切了六刀：



03
少发：缩小每轮请求的输入体积

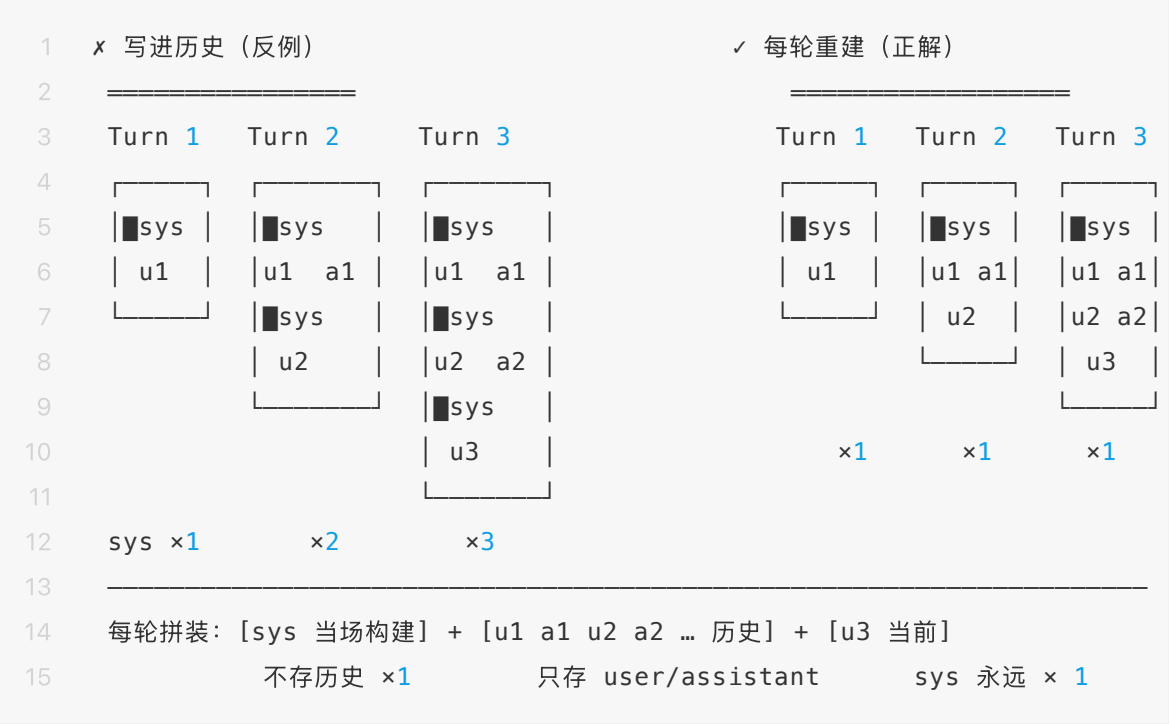
3.1 少发正确的过时的废话，随着模型进化 System Prompt

半年前，我们的 System Prompt 里一定会有这类内容：“你是一个 ReACT 循环的 Agent，每轮先观察、再思考、再执行工具，拿到结果后继续推理……”，有的版本还会附上完整的工具调用协议说明、错误重试策略、甚至 few-shot 示例来「教」模型怎么当 Agent。

这些在早期的确很有用，但现在最新的模型已经在海量 agent trace、工具调用日志和开源框架文档上训练过，ReACT 循环、tool_use 格式、多步规划这些范式早已内化。我们最初这类「教学文本」约占 1.2 K Token，删除后模型的工具调用准确率和任务完成率观测没有任何下降，但每轮请求少了这些部分，这部分乘以量级还是一个很大的 token 消耗量。

3.2 System prompt 每轮重建，不写进历史

缓存命中率优化



System Prompt 往往会占用 context 5%–%10（200K context window）,如果每轮都写进去就会浪费占用很大的 context window。因为这个优化是收益最高的，特别是 OpenAI 接口原生不支持 system 请求字段，全靠 agent框架和 agent 工程师自己处理。Anthropic 原生支持system 字段，只要将 System Prompt 放在 system 字段不会出现这个情况了

3.3 工具 schema 按需读取（Deferred Tools）

工具 schema 是每轮请求的固定token消耗。常驻工具加上多个 MCP server，完整工具占用 context window可能比 system prompt 还大，而模型并不会每次都会去调用 MCP。Anthropic 和 OpenAI 已原生支持这套语义。核心不变量：tools 数组每次请求字节相同（stub 只占 name），前缀缓存永远命中；完整 schema 经 `tool_search` 以 tool result 走 message 层注入，不碰 tools 数组。详见 Claude Code 官方博客。

但是百炼不支持。实测 Chat Completions 和 Responses API 均静默忽略 `defer_loading`、`tool_search` 字段（HTTP 200，工具按全量 schema 正常调用，无搜索流程）。因此我们参考 Claude Code 的思路，在 harness 层自行实现了等价机制：把低频工具的完整 schema 从每轮请求中移除，只在真正需要时加载。

我们把所有工具分成两个池子：



缓存命中率优化

```
┌──────────────────────────────┐      ┌──────────────────────────────┐
│  每轮携带完整 schema          │      │  只留一行目录在 prompt        │
└──────────────────────────────┘      └──────────────────────────────┘
   每次请求的 tools 数组（固定）            system prompt 内（跨 reveal 稳定
```

延迟池在 system prompt 中保留按名称排序的完整目录，最多 50 行，每行“工具名 — 一句话提示”。目录列出所有延迟工具，不论是否已 revealed；reveal 不从目录删除条目。激活路径有两条：

```
路径 A：模型主动搜索                    路径 B：Skill 激活自动扫描
====================                  ============================
模型看到能力目录                        ActivateSkill返回skill 正文
      |                                       |
ToolSearch(query)                      逐词匹配正文中的deferred工具名
      |                                       |
返回 reveal payload ──────┬──────── 命中的工具名
(name+schema+guidance)    |
                reveal payload 追加到history尾部，ToolRevealRecord 持久
                          |
                同一 AgentLoop 即可调用
                          ▼
                CallDeferredTool(真实工具名, 参数)，代理解析后走真实工具的完
```

对缓存的影响在于，工具召回不动前面任何内容：system prompt 不变、tools 数组不变，新工具的schema 只是追加在对话末尾。前面没变，缓存照常命中；同一个工具再搜一次，只回一句“之前已经加载”，不会重复塞入 schema。

效果：目录只有几十行文本，相比几十个完整 工具schema 动辄上万 token 的固定开销，不常用工具不再每轮占位。

3.4 常驻工具要标准化，工具的schema 也要精简

如同 System Prompt 优化一下，常见的工具无法 deferred（Read、Edit、Grep、Glob、Write、Bash 等工具），但是只要按照 claude code 工具定义，入参和出参和 claude code 保持一致，就可以尽可能的精简它的 schema。因为 claude code 作为最流行（没有之一）的 coding 工具，它的工具定义，调用，出入参早就被深深刻入的LLM 的训练中，只要模仿它，就能很大程度节约token，同时还能避免LLM 对工具的推理以及避免工具调用失败（这是最便捷和最高效的方式，如果自定义一个同名Read，然后出入参不一致，很有可能LLM第一次基会出错，这样浪费了 Agent Loop轮次，时间，又浪费token）。

3.5 Skills 去重：同一份只激活一次

一份 skill 正文动辄几千 token，如果连续 5 轮都激活同一个 skill，却每次都把全文重新塞进请求，就等于让同一份内容占用 5 次 token。重复得多，也不代表模型记得更牢。《Lost in the Middle》发现，模型利用长上下文中的信息存在明显的位置偏差：相关信息放在开

果更好，落在中间时表现会显著下降。这里更准确的说法不是“中间文字的注意力更低”，而是模型对中间信息的利用效果更差。反复塞入同一段长文本，既浪费 token，也不能保证效果更好。

所以我们让 skill 全文只注入一次，固定在历史中作为锚点；后续无论手动还是自动再次激活，都只在上下文尾部补一条短提醒，告诉模型上文中的 skill 仍然生效。

激活有两条路径，共享同一个单份不变式，同一 skill 的全文在任意一次请求的 messages 中至多出现一次：

```
通道 A：用户 /skill-name（metadata）          通道 B：LLM 调 ActivateSkill（
=====================================      ==============================
首次    全文注入该条 user msg（锚点）            首次    返回全文，tool result
重复    不重注入，仅附一行提示                   重复    幂等短回执 ~150 字符
        "指令已在上文，本轮仍生效"                       "指令已在上文，本轮仍生效"
锚点固定 → 成为稳定前缀 → 后续轮次缓存命中 ×10%
```

两条路径同时存在时，按照如下规则执行：谁先激活谁持有全文，后来的拿短回执；并且激活集合每轮从历史记录推导，不依赖内存状态，所以重启后结果一致。

Token 收益很直接。以前同一个 skill 激活 5 次，几千 token 的正文就会被塞进历史 5 遍；现在只塞第一遍，后面每次只补一句约 150 字符的提醒。按 5 次计算，光这部分就能少写接近 80%。

3.6 重复 Read 去重

模型在一轮 agent loop 里经常对同一个文件发起多次 Read，和上面的 skills 一样，如果重复读同一份，放在历史 message 中又是占用 token，浪费 token。 于是我们维护一个已读路径集合（通过文件系统的 mtime），在每批工具执行前检查：路径已读过、或同批次内重复 → 不执行，直接返回文件未改变，重复度。如果有改变，mtime 就会变，自然而然可以使用 read 工具。 同时及时真的想去读文件还是有很多 shell 命令，现在 LLM 都很聪明，它会用 sed,cat 等 shell 命令 读取文件。

```
同一 run 内（一条用户消息触发的 model↔tool 循环）
=================================================
Turn 1  Read("/src/main.rs")  → 正常返回全文（8000 字符）
                                路径记入已读集合
Turn 3  Read("/src/main.rs")  → 路径已存在，跳过
                                返回："Skipped duplicate readin the sa
Turn 5  Read("/src/main.rs", offset=100)  → 同样跳过（不看 offset）
wire 中：一份全文 + 两条 ~50 字符错误，而非三份全文。。
```

3.7 MCP 迁往 CLI + Skills：把 schema 从根上干掉

缓存命中率优化

Deferred tools 解决了「MCP schema 不每轮占位」，但 MCP 形态本身还有结构性成本：即使延迟加载，目录里每个工具仍要留一行条目，reveal 时完整 schema 至少付一次费；MCP 工具结果还是模型没法过滤的 JSON 块，返回多大就全量进历史。更根本的问题是：一个 MCP server 等于在运行时教模型一套新的接口格式，这份教学成本每次都按 token 付。

我们之前踩过一个坑。半年前，我们基于开源的 MongoDB MCP Server 封过一个查 Mongo 成本数据的能力，用途很窄，我们只注册了数据库类的 24 个工具，但是光这 24 个工具的描述文本就近一万字符、76 处参数注解，每轮两三千 token 的固定开销，而这只是一个 MCP server。能力越窄、server 越通用，浪费的比例就越夸张。

我们的做法是把这类能力迁到 CLI + Skills。回看开头的示意图，MCP 工具 schema 在固定前缀里占了一整块，迁移之后这块缩成 Bash 一个常驻工具：

```
MCP 形态                                    CLI + Skills 形态
================                            ============================
tools 数组: Bash + 几十个 MCP schema         tools 数组: Bash + 常驻工具 ← 字
system: 能力目录                             system: skills 目录 (CLI 名 +
用法: reveal 完整 schema → 调工具            用法: ActivateSkill 加载正文 → -
结果: JSON 全量进历史                        结果: head/grep/jq 过滤，只要需要
```

能力侧的 schema 开销归零，这是第 4 节「常驻工具要标准化」逻辑的延伸：模型天然擅长 CLI。预训练语料里有海量 Unix 材料，man page、--help 输出、shell 脚本，模型见过的 `find | grep | xargs` 远比任何一次 tools/call 多，怎么调、参数怎么拼、失败了怎么恢复，都是权重里现成的直觉，不用再放进请求。Unix 哲学也在降门槛：工具名即语义，不读 schema 也能猜个八九不离十；接口统一为文本流，一行管道就能拼出工作流；遇到陌生 CLI，跑一下 --help 自己就能摸索。自定义 MCP 恰恰相反，模型每次都要靠 schema 现学：一篇 arXiv 研究扫描了 103 个 MCP server 的 856 个工具，97.1% 的描述至少带一种质量坏味道，56% 连用途都没说清。

Skills 本身也是渐进式披露：触发条件、常用命令组合、避坑事项写进 skill 正文，按需加载，并复用第 5 节的全文去重机制。一个 CLI + 一份 skill = 一项完整能力：固定 schema 几乎为零，知识按需进入。还有两个附带收益。

一是输出可通过 shell 过滤。MCP 结果不可过滤、全量进历史，CLI 输出是文本流，head/grep/jq 只取需要的行——这是回传路径上的「少发」。

二是对缓存友好。MCP server 增删会合法地改变 tools 集合、炸掉前缀；CLI 形态下新增多少项能力，请求都不变一个字节。

常见对cli质疑声是 CLI 散落在每个终端，更新比集中部署的 MCP server 麻烦。其实恰恰相反：

1. 更新就是一条命令：`cli update all` 一次全升，还可以在服务端设置灰度名单。反观 MCP 集中式升级风险更大，新版本一发布所有客户端同时生效，发坏了没有缓冲地带。

缓存命中率优化

2. MCP更新反而有“历史对话污染”的包袱：调用会以带工具名和 JSON 参数的 tool_use 记录沉进历史，server 升级改名换参数后，模型照抄历史里的旧范式反复调错，新 schema 就在跟前他也不会用，除非显示特别声明。
3. CLI 形态没有这个包袱：历史里沉淀的是稳定的 shell 一行命令，演进通常向后兼容；skill 正文是用法的唯一事实源，`cli update skills` 一次整体替换，旧范式不残留。

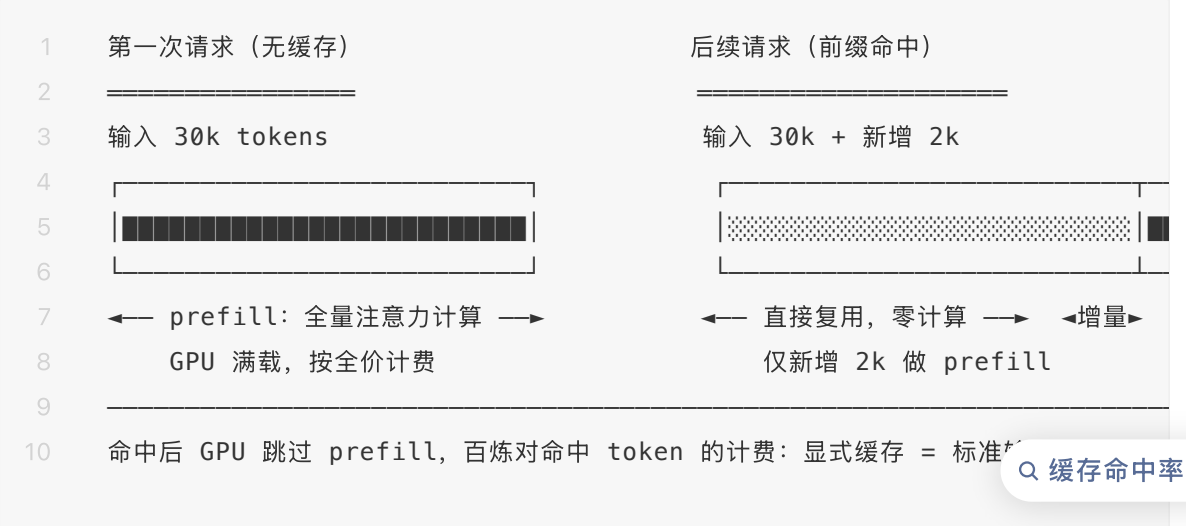
这也是为什么 CLI + skills 正在成为开源工具的标配，连 MCP 自己都在类 CLI 化。2026-07-28 版本的规范主线就是无状态化：退役 initialize 握手和 Mcp-Session-Id，每个请求自带协议版本和能力声明，像自带 flag 的 CLI 调用；tools/list 要求确定性排序并可声明 ttlMs 缓存，官方明说这是为了 prompt cache 稳定，正是本文第二节的那套纪律。Anthropic 则更直接，提出 Code execution with MCP：把 MCP server 暴露成可执行代码模块，agent 写代码组合调用，官方示例里单任务从 150k token 降到 2k（降幅 98.7%）。拆开看，这全是 CLI 化的套路：无状态调用、可组合、渐进式披露、过程不进上下文。

因此现在如需要新建一些能力，我们是否可以先想想「能不能是一个 CLI + 一份 skill」？

少发这章节远不止于此，本文只写了其中高收益的一部分。Agent loop 和周边生态里还有很多可以优化的地方，例如工具尽量并行、Skills refer 不常用内容等等，上一节的 MCP→CLI 迁移就是生态层面的一个实践。最后想强调一点：不要放弃任何一个字符的优化。发出去的内容都会乘以基数，固定前缀每轮全量重发，早进入历史的内容会伴随此后每一轮；前缀里砍掉 1k，30 轮会话就是 30k，哪怕命中缓存折扣，也还要付一成的价。少发无小事，每个字符都算数，乘上轮次就是真金白银。

04
发得便宜：让已发的尽量命中缓存折扣

少发 input 就少自然而然就能节约 token，但是也需要让发了的内容尽可能复用（复用 LLM 的 KV Cache），节约费用同时提速 LLM 推理。开始之前，我们需要了解： Transformer 生成每个 token 都要对前文所有 token 做运算。第一次收到输入时，模型对整段前缀逐层算出 Key / Value 矩阵(prefill)，这是推理中算力最密集的阶段；之后每生成一个 token，只需拿新 token 的 Query 去查已有 K/V，计算量极小。KV Cache 就是把 prefill 阶段的产物留在显存或磁盘上，下次请求只要前缀字节相同，即可跳过 prefill 直接复用。



```
隐式缓存 = 标准输入价 × 20%（无创建费，但无固定有效期，系统自动管理，命中不可预期
```

百炼显式缓存当前有四条关键规则：缓存内容至少 1,024 Token、单次最多 4 个标记、有效期 5 分钟且命中续期、Tools 定义也参与 system 前缀的缓存计算。官方也明确建议多轮对话把 `cache_control` 标记放在最后一条消息上。详见显式缓存最佳实践。

为什么不能全依赖隐式缓存？百炼官方声明不保证隐式缓存的命中率，且命中折扣（×20%）比显式（×10%）贵一倍。我们在 qwen3.7-max 上的同场景测试（详见第 2 节）验证了这一点：隐式加权命中率 0.800，但会出现单轮骤降至 0.00 的全量 miss，不可预测。显式缓存则是合约：打标记、前缀字节稳定，5 分钟内一定命中，同场景加权 0.898、无全量miss。用了显示缓存并不意味着隐式缓存并非不存在——显式标记 5 分钟 TTL 过期后，仍可能用隐式缓存兜住一部分请求，省下 ×20% 的命中费用。

综合下来，显式命中按 ×10% 计费且跳过 prefill，费用降到无缓存的约 12%，首 token 延迟随命中比例线性缩短。

但折扣有前提：前缀字节必须跨轮不变，变了一个字节，从那里往后的 KV 全部作废。于是乎我们需要在软件工程完成以下（不止这两）：

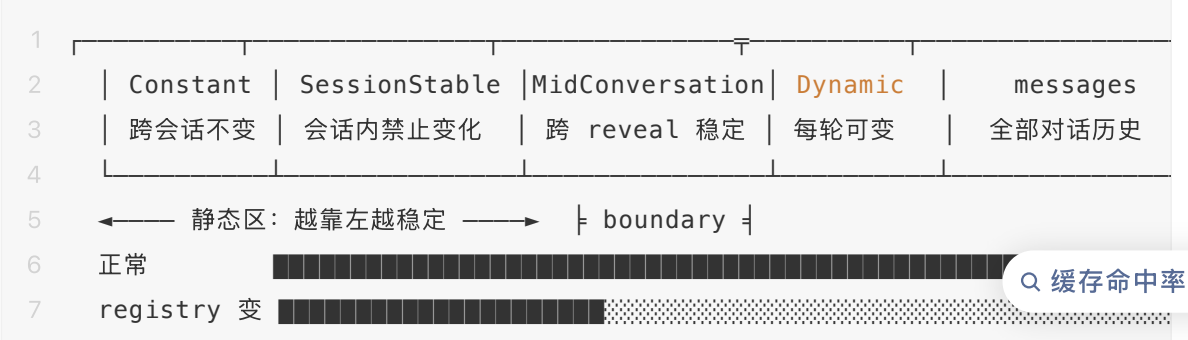
- 让前缀本身尽量稳定;
- 让断点标记覆盖尽可能长的前缀。

4.1 System prompt 按稳定性分层

「System prompt 不是不变的吗？」产品身份和固定规则确实不变，但工程往往 system prompt 还塞了 memory 索引、应用上下文、skills 列表这些变化频率不同的内容，它们共享同一段前缀字节。不分层，任何一处变化都让整段前缀连同后面几十 k 对话历史一起失去缓存。所以按变化概率从低到高排列，万一某层被迫变了，左侧前缀仍命中，只有右侧连同 messages 数组失效——控制爆炸半径。

用 `__SYSTEM_PROMPT_DYNAMIC_BOUNDARY__` 把 system prompt 切成静态区和动态区。静态区三档：

1. `Constant`：产品身份、固定规则。跨会话不变，发版才动；
2. `SessionStable`：AGENTS.md、skills 列表、应用信息。会话内禁止变化；
3. `MidConversation`：deferred tools 目录。跨 reveal 稳定——列出所有延迟工具，不因 reveal 删减或重排。



```
Session 变  ████████▒▒▒
```

2026-07-26 同场景案例对比（qwen3.7-max，同一个 prompt诊断类任务（三个 subagent 执行）；同一个 prompt，并且清空 memory ，两组非严格受控测）：

```
显式 ON · 13 calls                                    隐式 ON · 16 calls
turn    rate     tokens    latency                    turn    rate     tokens
──────────────────────────────────────────           ──────────────────────
 2     0.000     17.1K     12.0s   创建                2     0.000     17.6K
 3     0.670     25.2K      8.4s   Δ大                 3     0.630     27.2K
 4     0.640     39.4K     17.2s   Δ大                 4     0.000     44.1K
 5     0.983     39.8K     10.2s                       5     0.957     46.4K
 6     0.996     42.6K     60.2s   长生成              6     0.848     50.3K
 7     0.866     46.1K     10.5s                      11     0.208     71.7K
10     0.916     57.6K      4.1s   短生成             13     0.899     86.3K
13     0.987     68.6K    128.5s   长生成
加权 0.898 | 中位 0.939                              加权 0.800 | 中位 0.899
最低 0.640（无灾难脱靶）                             最低 0.000（两次灾难脱靶）
```

显示和隐式最明显差异是缓存命中下线。显式最低 0.64，是因为读了三个 subagent 的结果。隐式缓存也是读了三个 subagent 的结果则突然变成 0了。我去翻了下百炼官方文档，文档中明确说明不保证隐式命中率。

其实这一节就很简单，复用之前的，让模型少计算。

05
少想：让模型不要思考太多

最开始我们是在 prompt 里加一句“少想多干，不要过度分析”，想着模型应该就会收着点；结果发现没用。一个回答能思考突出 1w 多的个字符。其实原因也很直接：prompt 只是建议，模型觉得题目值得想时，还是会继续想。百炼按输出 Token 计费时，思考和最终答案都在输出侧账单里。所以后来我们不再劝模型少想，而是直接从请求参数上控制，根据场景直接控制。

5.1 不需要推理的调用，直接关掉

Agent 周围有很多 LLM 调用，但不是每一次都值得开思考。在我们的 agent 一次 prompt 对话中会有这些辅助的，但是会调用 llm 功能：标题生成、意图分类、查询富化、轮后建议、字段提取、简单压缩，这些功能都有几个共同点：输入不长，输出很短，答案空间也很窄，这类辅助调用在百炼请求里直接关闭 thinking。主 Agent Loop、需要做判断的 Subagent 才开启；关闭之后明显的Token 下降，首包也快了很多。

5.2 开了思考，也不能让它无限想

国产模型目前我们发现一个共性，都喜欢思考，并且思考链特别长。我们 agent 用的是 qwen3.7 max 国产模型，而且主 agent 推理的调用必然会开启 thinking，于是乎我们探索出一条路，

缓存命中率优化

给 thinking_budget 设置上限。这个参数表达的是「最多允许花多少思考 Token」，问题简单时它可以提前结束，问题复杂时到上限也必须转入答案，让它不会一直哪些光想不干活。

我们最后给主 Agent 和推理型 Subagent 选了 4,096，同时再给最终回答预留 4,096。这个数字没有通用性，不是百炼的推荐配置，是在我们故障诊断样本上扫出来的拐点：从不思考提高到 4K，工具选择和根因判断明显变好；继续放到 8K、16K，reasoning 越来越长，任务完成率却没有一起上涨，反而思考更长，整体的调用时间偏长。

```
一次模型调用的输出空间：允许推理，但不允许吃掉最终交付空间
┌──────────────────────────┬──────────────────────────┐
│ thinking 最多 4,096      │ 最终回答预留 4,096       │
└──────────────────────────┴──────────────────────────┘
```

06
SubAgent：探索过程不进主上下文，只回传结论

我们最开始没有 Subagent，查日志、翻监控、看发布记录、搜索代码，全由主 Agent 自己做。问题很快就暴露出来：真正有用的结论可能只有一句，但为了得到这句话，前面查过的几百行日志、十几次搜索结果和几条走错的路，而且上下文窗口动不动就被塞满，然后 compact。 在故障诊断这个场景，时间就是金钱，一旦进入 compact 就以为又会浪费好几分钟；后面我们就引入了 subagent，让它只带结论回来。

```
所有事情都在主 Agent 里                把探索交给 Subagent
═══════════════════                    ═══════════════════
主上下文                               Subagent 独立上下文
┌──────────────────────┐               ┌──────────────────────┐
│ 任务 + 原有历史      │               │ 一份独立的任务说明   │
│ 日志、搜索、失败尝试 │               │ 日志、搜索、失败尝试 │
│ 200k 探索过程        │               │ 200k 探索过程        │
│ 最终结论 0.5k        │               └──────────┬───────────┘
└──────────────────────┘                          │ 只回传结论
          │                            主上下文   ▼
后续每轮重发 200k                      ┌──────────────────────┐
                                       │ 原有历史 + 0.5k 摘要 │
                                       └──────────────────────┘
```

新 Subagent 启动拿到自己的 system prompt 和主 Agent 写的一份任务说明，然后在独立的 Agent Loop 里完成调查。任务结束后，完整结果落到文件里；主 Agent 默认只收到不超过 600 个字符的摘要和结果路径，需要复核证据时才按路径读取。

6.1 subagent 派发规则

判断要不要派 Subagent的原则，就是会不会产生大量主 Agent 后面用不到的过程。

缓存命中率优化

比如一次故障排查，子任务为了定位一条异常日志，前后读了 200k Token，最后只需要告诉主 Agent“问题从 10:32 的发布后开始，证据在这三个位置”。如果主 Agent 后面还要跑 10 轮，不做隔离，这 200k 探索记录就要跟着重发近 10 次；隔离以后，每轮只需要带着几百 Token 的结论。

```
主上下文少重发的 Token ≈ 父级剩余轮数 ×（探索过程 − 回传摘要）
```

但如果只是查一次 Pod 状态，工具调用只有 2k Token，查完主 Agent 就能回答，单独拉起一个 Subagent 反而更贵：它也有自己的 system prompt、工具目录和调用轮次，还多了一次委派与回传。

并不是并发 agent 就会自动省 Token。三个 Subagent 同时调查，只是等待时间变短了，三份上下文和工具调用仍然都要付费。只有几个方向互不重叠、最后又都能压成短结论时，才适合一起派出去。

最终原则就是四条，最好实现一个下发 agent 的规则，然注入 system prompt 中，现在 agent 能力很强，也很喜欢调用 subagent，我们只需要将规则写完：

适合拆出去	留在主 Agent
查多个目录、日志或数据源	一次工具调用就能回答
日志、指标、变更三个独立方向	下一步完全依赖上一步结果
过程很长，最后只需要结论和证据位置	主 Agent 最终仍要通读全部原文
只读探索，边界容易划清	多个人要同时修改同一份文件或状态

6.2 把任务交代清楚

委派时必须把任务写成一份能独立执行的说明。我们的工具会直接提醒主 Agent：Subagent 看不到当前会话（其实有的subagent 设计是允许 subagent 去看主 agent 的历史，但是通常不会这么设计），通常如下：

```
背景      为什么要查，已经出现了什么现象
目标      最终只需要回答哪几个问题
范围      可以看哪些目录、应用、时间段和数据源
已知事实  哪些已经确认，哪些方向已经排除
边界      只读还是可写，哪些文件和外部操作不能碰
返回      结论、证据位置、未确认项，以及做过的验证
```

结果也需要给 subagent 说需要什么样，不然Subagent 跑了 100k Token，最后回传一篇 20k Token 的完整报告，看起来很负责，这样隔离的作用就无效。我们实际只让结果回传这些内容：

```
状态      完成、部分完成，还是被阻塞
```

缓存命中率优化

```
结论          几句话说清发现了什么
证据          文件、行号、时间范围或查询条件
未确认项      哪些只是推测，哪些数据源还没查
下一步        主 Agent 还需要做什么
结果路径      完整日志和详细报告存在哪里
```

完整日志、搜索结果和中间表都留在文件里。主 Agent 先根据短摘要继续推理；只有某条结论真的会影响最终判断时，才顺着证据位置读取原文。文件是冷存储，摘要才是进入主上下文的热数据。

在Subagent 超时、预算耗尽或权限不足时，我们会用 hook 强制注入一轮让它自总结，这轮总结里面要求必须明确说自己查了哪些、没查哪些。

07
Hook：在循环边界拦截，避免无效轮次

hook 是成本优化的最后一关，同时也是让 agent 更高效的关键一步。因为模型输入是不稳定，随机的，是经常不按照要求输出内容，例如模型模型调用工具格式有时候会泄露不按照标准的 OpenAI Function tool 格式，空回答等等。于是乎，我们就需要定义一些 hook，这些 hook 都是根据各个模型的输出规则，可以从 github 各家的 issue 上面抓，很多都是通用的；还有一些得根据业务实测，会在运行过程中产生。这就需要我们最开始说的：建立度量标准来捕获这些异常案例。

我们线上见过几种很典型的 Agent 空转，一条 SSE 里同一句话重复几百次；连续几个 Turn 输出几乎一样的内容；工具虽然一直在调用，但每轮都在执行同一条命令，或者换着工具返回同样的空结果。从界面上看 Agent 还在工作，实际上已经卡死。更麻烦的是，这些重复内容会进入历史，下一轮再作为输入发给模型，越转越贵。靠 Prompt 提醒「不要重复」没有用，模型已经进入退化状态时，必须由 Agent Loop 在外面把它停下来。

7.1 一条 SSE 内重复

我们会边接收 SSE，边按句子检查新增内容。代码块里的文本和「好的」这类短句不参与判断；同一个正常句子在一条回复里完整出现第三次，就认为模型进入了生成循环，立即停止继续接收。

这个 Hook 来源于 DeepSeek flash 0731 模型：模型先正常回答，随后把“日志可达，定向查关键字”一类句子重复了 685 次，并且没有提示，685 是因为手动 abort 了。所以我们在流式阶段做了 hook，按照段，句为分割进行 hash。第三次重复时就断掉，hook 返回你一直在重复，继续完成 prompt。重新回答后仍然复读，就直接结束这次 Loop，不再无限重试。

7.2 多个 Turn 说同样的话或者一直重复调用同样的工具

这个 hook 几乎所有百炼自部署的模型都遇到过，有时单次响应看起来正常，但 Agent 每一轮都在输出同一段分析。我们会给回答前 500 个字符生成指纹，与最近三个 Turn 比较

缓存命中率优化

80%，就认为它没有产生新进展。

第一次、第二次发现重复时，Runtime 会提醒模型换一种办法、换工具，或者直接向用户确认。连续第三次仍然重复，就结束循环。只要回答发生变化，重复计数就会清零，然后重新计数。

7.3 一直调工具但是无效

只看文本还不够，Agent 也可能用工具把自己伪装得很忙。我们又加了几种检查：

- 同一组工具和参数连续出现 8 个 Turn，先提醒它重新判断；提醒后仍不改变，就结束。
- 连续 5 个 Turn 的工具结果全部报错或为空，即使每轮换了不同工具，也会要求它更换思路；继续没有结果就结束。
- Shell 命令累计失败 4 次，会提醒它别再尝试同一路径，改用其他工具或已有证据回答。
- 用户连续 3 次没有响应工具确认，说明人已经离开，停止继续发起需要确认的动作。

7.4 空回答和假动作也不能进入历史

这个 hook 也是几乎所有百炼自部署的模型都遇到过：模型偶尔只输出 reasoning，没有正文和工具调用；也可能连续写“让我查询”“我再确认”，却始终没有真的执行。我们把这类结果当成无效回答，不作为最终内容保存，而是要求它直接给通过 hook 拦截注入让调用工具。如果最多纠正三次，第四次仍然为空就停止这轮对话，说明模型已经进入死循环了，在这轮对话中无法恢复。

最后我们还有一道兜底。如果内容变成大段 Prompt 复述、连续的自我指令，或者非流式场景漏过来的句子复读，也会被丢弃然hook 注入生成用户可读的总结，避免一篇明显退化的内容被当成正常答案交给用户。

所有 hook的核心都是纠正模型的行为，避免无意义的空转，或者无意义的内容，这样不仅节约了 token ，省钱的同时更提升了效率。

08
模型分流：不是每个任务都需要最贵的模型

前面所有优化都在同一个模型上展开。但回头看，agent 一次完整任务里的 LLM 调用，真正需要重推理的只有主循环和核心 subagent 的对话轮次。标题生成、意图分类、查询富化、建议生成、内存刷写，这些辅助调用的输入输出都很短，逻辑也很简单，用旗舰模型做这些事就像开卡车送信封。我们的做法是角色路由。`ModelRegistry` 里设了四个角色指针，每个指向一个具体的模型 alias，后端通过 gRPC 下发，客户端缓存到磁盘。所有 LLM 调用不直接指定模型名，而是通过角色指针间接引用，后端换模型不需要客户端发版。

```
ModelRegistry
├─ default_chat_alias ── 旗舰模型 · 主循环 / Subagent
├─ classify_alias ─────── Flash · 分类 / 标题 / 富化
├─ responses_alias ────── 场景模型 · Responses 工具
```

缓存命中率优化

```
└─ file_extract_alias ── 长文本模型 · 文件提取
```

`classify_alias` 承担几乎所有非核心调用：意图分类、查询富化、标题和建议生成、内存更新、质量评估首轮，以及 context 压缩降级。这些任务输入短、输出短、答案范围也窄，交给 Flash 模型即可，成本通常只有旗舰模型的 1/10 到 1/20。只有质量评估出现自相矛盾（`dimension_consistent == false`）时，才升级到旗舰重新判断。

Subagent 也按角色分流：`explore`、`data-query` 走 `classifyAlias`；`general-purpose`、`plan`、`verification`、`code-review` 走 `defaultChatAlias`。钉钉的诊断子任务大多只是“查一个指标，返回数据”，没有显式指定时也默认走 Flash。

依据业务场景让某些 agent 高度模式化直接使用轻量模型，例如我们的 K8s Agent 更彻底：四个角色全部指向 Flash。查 Pod、事件、日志和 Prometheus 指标都是高度模式化的工作，不需要旗舰模型的通用推理，单次诊断费用因此降到约 1/15。

最后用户在前端切换聊天模型时是主对话模型，不可选择classify_alias，让连标题、分类等后台调用的成本变高。

模型路由目的是让简单的活用便宜模型改，杀鸡不用牛刀；这样简洁也节省很多 token 费用，在 agent turn 周边有很多操作本身就是一次性且简单的，因此用更轻量的模型反而更快更高效。

09
88% 降幅怎么算的

最后我们只需要将前面两把最大的杠杆放到一起算一次：裁剪减少实际发送量，缓存降低重复部分的单价。

假设一次会话跑 20 轮，每轮输入都是 $S + i \times \Delta$：S 是 system、tools 等固定前缀，Δ 是每轮新增的对话和工具结果。无缓存时，20 轮一共发送：

$$20 \times S + (1 + 2 + \dots + 20) \times \Delta = 20S + 210\Delta$$

显式缓存还要区分两种价格：第一轮和每轮新增的 Δ 按创建价 ×125% 计算，之前已经缓存的前缀按 ×10% 计算。把费用统一折算成“按标准输入价购买了多少 Token”，结果如下：

20 轮会话 · 标准输入价 ¥12/M Token

场景	实际发送	等价全价 Token	输入费用	降幅
无优化	820k	820k	¥9.84	–
仅加缓存	820k	151k	¥1.81	81.6%
裁剪 + 缓存	532k	96.9k	¥1.16	88.2%

缓存命中率优化

只加缓存时，实际仍然发送 820k Token，只是大部分按折扣计费；再做裁剪，发送量降到 532k，最终费用约为原来的 11.8%。所以 88% 不是某一个优化单独带来的，而是“少发”和“发得便宜”叠在一起的结果。这个数字是统一假设下的测算，只算主 Agent 输入侧，并假设 20 轮都在 5 分钟 TTL 内连续命中。

如果算上其他优化，远远不止这个数字。

10
结语

回头看，成本优化并不是一套外挂式的 Token 优化技巧，它们就是 Harness 实践。模型只负责生成，Harness 决定哪些内容要发、哪些过程要隔离、什么时候该停，以及这次任务应该用什么模型，最终怎么高效完成这个 prompt。

Agent Loop 的成本本身也没有什么单一解法。前缀少一点、缓存多命中一点、探索少带回来一点、无效轮次少跑一次，单看都不惊人；但这些开销会在每一轮、每一次任务里重复，最后的差距就是这样乘出来的。

这就是开头提示写出一个能跑的 Agent 很容易。但是需要让它长期跑得起，跑得高效，才是 Harness 真正要解决的问题。

**作者注：本文在 @曹建新 指导下，idea 来源 @陈威， 实践过程中得到 @马从东 @李德帅 @黄孟军 @冯松 @李国标 @胡盛 的帮助指导和支持。*

参考资料：

[01] 百炼上下文缓存

https://help.aliyun.com/zh/model-studio/context-cache

[02] 百炼显式缓存最佳实践

https://help.aliyun.com/zh/model-studio/explicit-cache-guide

[03] 百炼模型调用价格

https://help.aliyun.com/zh/model-studio/model-pricing

[04] DashScope API 参考：思考模式与 thinking_budget

https://help.aliyun.com/zh/model-studio/qwen-api-via-dashscope

[05] 百炼 OpenAI 兼容接口：reasoning_effort 映射

https://help.aliyun.com/en/model-studio/qwen-api-via-openai-chat-completions

[06] Claude Code: Prompt Caching Is Everything

https://claude.com/blog/lessons-from-building-claude-code-prompt-caching-is-everything

[07] Anthropic: Advanced Tool Use（defer_loading beta）

https://www.anthropic.com/engineering/advanced-tool-use

[08] Azure OpenAI: Tool Search with Responses API

https://learn.microsoft.com/en-us/azure/foundry/openai/how-to/tool-search

[09] MCP 2026-07-28 规范发布公告

https://blog.modelcontextprotocol.io/posts/2026-07-28/

[10] MongoDB MCP Server（开源，cost-epang 案例来源）

https://github.com/mongodb-js/mongodb-mcp-server

[11] MCP Spec 2026-07-28: Server Tools

https://modelcontextprotocol.io/specification/2026-07-28/server/tools

[12] Anthropic: Code execution with MCP

https://www.anthropic.com/engineering/code-execution-with-mcp

[13] Anthropic: How we built our multi-agent research system

https://www.anthropic.com/engineering/multi-agent-research-system

[14] Anthropic: Effective context engineering for AI agents

https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents

[15] MCP Tool Descriptions Are Smelly! (arXiv 2602.14878)

https://arxiv.org/abs/2602.14878

[16] Lost in the Middle: How Language Models Use Long Contexts (Liu et al., TACL 2024)

https://aclanthology.org/2024.tacl-1.9/



缓存命中率优化

欢迎留言一起参与讨论~

阅读 6026

留言 3

写留言



Chord 浙江 9月12日

这是真大佬



kk 湖北 8月21日

来试试



煎bingo子 上海 8月21日

Context is all you need!

首评

缓存命中率优化